جزوه دوره مهندسی داده و مهندسی هوش مصنوعی

Data Engineering & AI Engineering

یادداشت‌های آموزشی، مفاهیم فنی و تمرین‌های دوره



جلسه ششم:SQL (قسمت اول)
مدرس :کوثر اصغری
تاریخ :سه‌شنبه ۵ آبان ۱۴۰۵

تاریخ به روز رسانی : 20261028

درباره این جزوه

این جزوه با هدف ثبت، سازمان‌دهی و بازنویسی آموزشی مطالب دوره رایگان «مهندسی داده و مهندسی هوش مصنوعی» تهیه شده است.

محتوای آن بر اساس توضیحات مدرس، مثال‌های مطرح‌شده در کلاس و پرسش‌وپاسخ‌های آموزشی تدوین می‌شود. تلاش شده است مطالب گفتاری کلاس به متنی منظم، قابل فهم و مناسب مطالعه مستقل تبدیل شوند، بدون اینکه مفهوم اصلی صحبت‌های مدرس تغییر کند.

هدف این جزوه صرفاً خلاصه‌کردن جلسات نیست؛ بلکه ایجاد یک منبع آموزشی است که دانشجو بتواند در طول دوره و حتی چند ماه پس از پایان آن، برای یادگیری دوباره مفاهیم، مرور نکات فنی و آماده‌شدن برای فعالیت‌های عملی به آن مراجعه کند.

رویکرد آموزشی و شیوه تدوین

در تدوین مطالب، ابتدا مفاهیم اصلی هر جلسه با زبانی ساده و دقیق توضیح داده می‌شوند. سپس مثال‌های مدرس، کاربردهای عملی و ارتباط موضوعات با مسائل واقعی مهندسی داده و هوش مصنوعی بررسی می‌شوند.

برای حفظ دقت علمی و تمایز میان محتوای اصلی کلاس و توضیحات تکمیلی، از ساختار زیر استفاده شده است:

- **مطالب اصلی جلسه :**بازنویسی منظم توضیحات مدرس با حفظ مفهوم و مثال‌های اصلی.
- **تکمیل فنی :**توضیحات دقیق‌تر، اصلاح برداشت‌های احتمالی و نکات تکمیلی بر اساس منابع فنی معتبر.
- **مثال اضافی :**مثال‌ها و سناریوهایی که برای درک بهتر مفاهیم به جزوه افزوده شده‌اند.
- **نکات کاربردی :**ملاحظات اجرایی، محدودیت‌ها و تجربه‌های مرتبط با استفاده عملی از مفاهیم.
- **جمع‌بندی و سوال‌های مرور :**مرور مهم‌ترین مطالب و ارزیابی میزان یادگیری در پایان هر جلسه.

اصطلاحات تخصصی مانندData Pipeline ، Data Warehouse، ETL، Orchestrationو سایر مفاهیم فنی با نام اصلی انگلیسی حفظ شده‌اند تا مطالعه مستندات رسمی و منابع تخصصی برای دانشجویان آسان‌تر باشد.

محدوده و توسعه جزوه

این مجموعه به‌صورت تدریجی و هم‌زمان با انتشار جلسات جدید تکمیل می‌شود و دو مسیر اصلی دوره، یعنی Data Engineering وAI Engineering ، را پوشش می‌دهد.

نسخه‌های به‌روزشده جزوه‌ها، منابع تکمیلی و یادداشت‌های هر جلسه از طریق مخزن عمومی GitHub در دسترس خواهند بود.

مخزن جزوه‌ها:

https://github.com/LaneZero/data-ai-engineering-course

وب‌سایت اصلی دوره:

https://mostafaghadimi.github.io/data-course/

یادداشت درباره اعتبار محتوا

این جزوه یک مجموعه آموزشی مستقل و غیررسمی است و وابستگی رسمی به مدرس یا برگزارکنندگان دوره ندارد.

حقوق محتوای اصلی دوره متعلق به صاحبان آن است. توضیحات تکمیلی و مثال‌های افزوده‌شده نیز لزوماً بیانگر دیدگاه یا تأیید مدرس نیستند.

با وجود تلاش برای حفظ دقت فنی، توصیه می‌شود هنگام اجرای پروژه‌های واقعی، اطلاعات حساس یا تصمیم‌گیری‌های فنی مهم، مستندات رسمی ابزارها و منابع اصلی نیز بررسی شوند.

هدف از انتشار این مجموعه، **کمک به یادگیری عمیق‌تر، مرور منظم مفاهیم و اشتراک دانش با علاقه‌مندان حوزه داده و هوش مصنوعی** است.

فهرست مطالب

۱. جایگاه جلسه و اهداف یادگیری ۱

۲. چرا SQL به وجود آمد؟ ۱

۲.۱ مسئله فروشگاه و فایل متنی ۱

۲.۲ SQL زبانی توصیفی است ۲

۳. جدول رابطه‌ای: ردیف، ستون و کلید ۳

۳.۱ NULL به معنای چه چیزی است؟ ۳

۴. خانواده‌های اصلی دستورات SQL ۴

۴.۱ DDL: تعریف جدول و محدودیت‌ها ۴

۴.۲ DML: افزودن، ویرایش و حذف ردیف ۵

۴.۳ DELETE, TRUNCATE و DROP ۵

۴.۴ DCL و TCL: دسترسی و تراکنش ۶

۵. خواندن داده با SELECT ۷

۵.۱ ساختار یک Query ساده ۷

۵.۲ فیلترها: AND, OR, IN, BETWEEN و LIKE ۷

۶. توابع تجمیعی و گروه‌بندی ۸

۶.۱ COUNT, SUM, AVG, MIN و MAX ۸

۶.۲ GROUP BY: هر کشور چند مشتری دارد؟ ۸

۶.۳ HAVING: فیلتر پس از گروه‌بندی ۹

۷. ترتیب نوشتاری و ترتیب منطقی پردازش ۹

۷.۱ CASE و COALESCE برای داده خواناتر ۱۰

۸. JOIN: ترکیب جدول‌ها و مسئله معنای ردیف ۱۱

۸.۱ چرا جدول‌ها را جدا ذخیره می‌کنیم؟ ۱۱

۸.۲ مثال مشتری و سفارش ۱۲

۸.۳ یافتن مشتریان بدون سفارش ۱۲

۸.۴ چرا پس از JOIN شمارش را دوباره بررسی می‌کنیم؟ ۱۳
۹. موارد اشاره‌شده برای ادامه یادگیری ۱۴
۱۰. Workflow تمرین و چک‌لیست ۱۴
۱۱. جمع‌بندی ۱۴
۱۲. سوال‌های مرور ۱۴
۱۳. پاسخ‌نامه مختصر ۱۵
۱۴. واژه‌نامه ۱۶
۱۵. انتقال تجربه مدرس ۱۷

۱. جایگاه جلسه و اهداف یادگیری

این جلسه نخستین ورود جدی دوره به SQL است. مدرس با مسئله نگهداری سفارش‌ها در فایل متنی شروع کرد، توضیح داد چرا DBMS و زبان پرس‌وجوی SQL شکل گرفتند و سپس با داده‌های آموزشی یک فروشگاه، ساختار جدول، خانواده دستورها، فیلتر، تجمیع و اتصال جدول‌ها را تمرین کرد. هدف این جزوه صرفاً حفظ دستورها نیست؛ باید بفهمیم هر دستور روی چه داده‌ای عمل می‌کند، نتیجه هر سطر چه معنایی دارد و چرا تعداد ردیف‌ها ممکن است بعد از JOIN تغییر کند.

پس از مطالعه بتوانید یک Query پایه بنویسید؛ تفاوت DDL, DML, DQL, DCL و TCL را توضیح دهید؛ `WHERE`، `GROUP BY` و `HAVING` را تشخیص دهید؛ وضعیت `NULL` را درست بررسی کنید؛ و خروجی JOIN را از نظر تعداد و معنای ردیف‌ها تحلیل کنید.

۲. چرا SQL به وجود آمد؟

۲.۱ مسئله فروشگاه و فایل متنی

مدرس فروشگاهی کوچک را مثال زد که سفارش‌هایش را در یک فایل متنی ذخیره می‌کند. هر خط شامل شناسه سفارش، نام مشتری، مبلغ و تاریخ است. اگر بخواهیم بدانیم «سارا چقدر خرید کرده؟»، برنامه‌ای لازم داریم که فایل را باز کند، خط‌ها را جدا کند، نام مشتری را تشخیص دهد، مبلغ‌ها را به عدد تبدیل کند و جمع بزند. برای سوال بعدی، مثلاً «پرفروش‌ترین روز کدام بوده؟»، باید منطق تازه‌ای بنویسیم. در این الگو، برنامه‌نویس مسئول جزئیاتی می‌شود که بهتر است یک سامانه مدیریت داده بر عهده بگیرد.

مشکل‌ها فقط طولانی‌شدن کد نیستند: خواندن پرهزینه اطلاعات، تغییر هم‌زمان فایل توسط چند نفر، خراب‌شدن داده در اثر قطعی هنگام نوشتن، نبود کنترل‌های کیفیت، دشواری برقرارکردن رابطه میان فایل سفارش و مشتری و دسترسی‌های بسیار کلی به کل فایل، همگی در کلاس مطرح شدند.

دردسر در الگوی فایل ساده	پاسخ معمول در DBMS
نوشتن برنامه برای هر پرسش	اجرای Query با زبان SQL
جست‌وجوی پرهزینه در حجم زیاد	Index، بهینه‌سازی و روش‌های مناسب دسترسی
نوشتن هم‌زمان و ناسازگاری	Transaction و کنترل هم‌زمانی
نوشته‌شدن ناقص یا داده نامعتبر	Transaction و Constraint
رابطه نامشخص بین فایل‌ها	جدول، Key و JOIN
اشتراک‌گذاری همه اطلاعات یک فایل	User, Role و مجوزهای تفکیک‌شده

- **تکمیل فنی:** SQL خودِ پایگاه داده نیست؛ زبان تعریف و پرس‌وجوی داده است. DBMS نرم‌افزاری است که ذخیره‌سازی، بازیابی، کنترل دسترسی و تراکنش‌ها را مدیریت می‌کند. سیستم‌های فایل و قالب‌های جدید نیز می‌توانند هم‌زمانی و جست‌وجوی پیشرفته داشته باشند؛ مقایسه کلاس با یک فایل متنی ساده است.

۲.۲ SQL زبانی توصیفی است

در SQL معمولاً می‌گوییم **چه نتیجه‌ای** می‌خواهیم؛ موتور پایگاه داده تصمیم می‌گیرد **چگونه** آن را تهیه کند. مثلاً می‌نویسیم مشتری‌های ایران را برگردان، نه اینکه فایل را از کدام بلوک دیسک باز کند و چند بار روی آن حرکت کند. موتور می‌تواند از Scan یا Index و یک Execution Plan مناسب استفاده کند. پاسخ معمول Query از دید کاربر در قالب مجموعه‌ای از ستون‌ها و ردیف‌ها نمایش داده می‌شود.

مدرس به پیدایش مدل رابطه‌ای در IBM، نام اولیه SEQUEL و استانداردشدن تدریجی SQL اشاره کرد. دلیل ماندگاری این زبان، کاربرد آن در ابزارها و پایگاه‌های داده متعدد و نقش مشترکش در فعالیت‌های تحلیلی و مهندسی داده است.

- **تکمیل فنی:** SQL استاندارد مشترک دارد، اما نحو و قابلیت‌ها در PostgreSQL, SQL Server, MySQL و دیگر سامانه‌ها کاملاً یکسان نیستند. `LIMIT` نمونه‌ای از تفاوت‌هاست: در T-SQL رایج است از `TOP` یا `OFFSET ... FETCH` استفاده شود. کدهای جزوه در بخش خواندن داده، با الگوی PostgreSQL سازگار نوشته شده‌اند؛ ساختار و داده نمونه کلاس به‌صورت فایل مستقل در اختیار این جزوه نیست.

۳. جدول رابطه‌ای: ردیف، ستون و کلید

مدرس دیتاست فروشگاهی شامل `customers`، `orders` و `products` را نمایش داد. در `customers`، هر ردیف یک مشتری است؛ ستون‌ها ویژگی‌هایی مانند نام، ایمیل، شهر و کشور را نگه می‌دارند. در `orders`، هر ردیف یک سفارش است و یک مشتری ممکن است چند سفارش داشته باشد. این تفاوت، پیش‌نیاز فهم JOIN است.

جدول	هر ردیف نشان‌دهنده	شناسه اصلی نمونه
`customers`	یک مشتری	`customer_id`
`orders`	یک سفارش	`order_id`
`products`	یک محصول	`product_id`

شناسه اصلی یا Primary Key باید مقدار یکتا و غیر NULL داشته باشد. Foreign Key رابطه میان دو جدول را مشخص می‌کند؛ مثلاً `orders.customer_id` به مشتری مربوط اشاره می‌کند. مدرس تأکید کرد نباید صرفاً از نام ستون حدس قطعی بزنیم که Key چیست؛ مفهوم Key باید در طراحی پایگاه داده تعریف شده باشد.

۳.۱ NULL به معنای چه چیزی است؟

در نمایش نمونه، برخی مقدارهای شهر یا کشور `NULL` بودند. `NULL` نماینده فقدان یک مقدار شناخته‌شده در آن خانه است؛ با عدد صفر، رشته خالی یا متن `Unknown` یکسان نیست. این تفاوت هم در فیلترها و هم در محاسبه و دسته‌بندی اهمیت دارد.

```
SELECT
    customer_id,
    full_name,
```

```
    city
FROM
    customers
WHERE
    city IS NULL;
```

- **تکمیل فنی:** شرط `city = NULL` برای یافتن مقدارهای NULL درست نیست. در SQL منطق سه‌ارزشی وجود دارد و مقایسه با NULL معمولاً نتیجه `UNKNOWN` می‌دهد. برای بررسی نبود مقدار از `IS NULL` یا `IS NOT NULL` استفاده کنید؛ تابع `COALESCE` در نمایش یا محاسبه می‌تواند مقدار جایگزین بدهد، بدون آنکه خود داده مبدا تغییر کند.

۴. خانواده‌های اصلی دستورات SQL

مدرس دستورات را در پنج خانواده معرفی کرد. این تقسیم‌بندی آموزشی کمک می‌کند بدانیم در هر موقعیت با ساختار پایگاه داده کار داریم، با مقدارهای داخل جدول، با خواندن داده، با مجوز یا با تراکنش.

خانواده	معنی	دستورهای نمونه	کاربرد
DDL	Data Definition Language	`CREATE`، `ALTER`، `DROP`، `TRUNCATE`	تعریف یا تغییر ساختار
DML	Data Manipulation Language	`INSERT`، `UPDATE`، `DELETE`	تغییر داده‌های جدول
DQL	Data Query Language	`SELECT`	خواندن داده
DCL	Data Control Language	`GRANT`، `REVOKE`	مدیریت مجوز
TCL	Transaction Control Language	`BEGIN`، `COMMIT`، `ROLLBACK`	مدیریت تراکنش

- **تکمیل فنی:** این پنج‌گانه، دسته‌بندی متداول آموزشی است؛ برخی مراجع `SELECT` را زیرمجموعه DML می‌دانند و جایگاه `TRUNCATE` در سامانه‌ها یکسان نیست. نحوه Rollback شدن DDL، به‌ویژه `TRUNCATE`، به محصول و شرایط تراکنش بستگی دارد و نباید آن را مطلقاً برگشت‌ناپذیر معرفی کرد.

۴.۱ DDL: تعریف جدول و محدودیت‌ها

مدرس ایجاد جدول محصول با ستون‌های شناسه، نام و قیمت را مثال زد. همراه تعریف ستون‌ها می‌توان قاعده‌های داده را نیز ثبت کرد: قیمت غیرمنفی باشد، شناسه تکراری نباشد و ستون اجباری خالی نماند.

```
-- Example added for practice; not a verbatim classroom command
CREATE TABLE products_demo(product_id INTEGER PRIMARY KEY,
```

```
product_name VARCHAR(120) NOT NULL, unit_price NUMERIC(12, 2) NOT NULL
CHECK(unit_price >= 0));
```

> **مثال اضافی:** در جدول آزمایشی بالا، درج قیمت منفی توسط `CHECK` رد می‌شود. بهتر است چنین آزمایش‌هایی را در پایگاه داده تمرینی اجرا کنید؛ دستورهای `ALTER` و `DROP` ممکن است ساختار یا داده‌ای را که برنامه‌های دیگر استفاده می‌کنند تغییر دهند.

مدرس `ALTER` را برای تغییر ساختار جدول موجود و `DROP` را برای حذف خود جدول معرفی کرد. تغییر ساختار در محیط عملیاتی باید با بررسی وابستگی برنامه‌ها، مجوز اجرا و برنامه بازگشت انجام شود.

۴.۲ DML: افزودن، ویرایش و حذف ردیف

`INSERT` ردیف جدید می‌سازد، `UPDATE` مقدارهای ردیف‌های موجود را تغییر می‌دهد و `DELETE` ردیف حذف می‌کند. مدرس تأکید کرد پیش از اجرای تغییر روی محیط واقعی، به‌ویژه به شرط `WHERE` دقت کنید.

```
-- Run only on a disposable demo table
INSERT INTO products_demo(product_id, product_name, unit_price)
VALUES
    (61, 'Training Item', 250.00);
UPDATE products_demo
SET
    unit_price = 275.00
WHERE
    product_id = 61;
DELETE FROM products_demo
WHERE
    product_id = 61;
```

- **تکمیل فنی:** `UPDATE` یا `DELETE` بدون `WHERE` می‌تواند همه ردیف‌های واجد شرایط جدول را تغییر دهد یا حذف کند. حتی با `WHERE` نیز بهتر است ابتدا همان شرط را با `SELECT` وارسی کنید و در محیط مناسب از Transaction، بکاپ و مجوز محدود استفاده شود.

۴.۳ DELETE, TRUNCATE و DROP

این سه دستور در ظاهر می‌توانند «چیزی را پاک کنند»، اما خروجی یکسانی ندارند.

دستور	آنچه حذف می‌شود	آیا جدول باقی می‌ماند؟	شرط ردیفی
`DELETE`	ردیف‌های انتخابی یا همه ردیف‌ها	بله	`WHERE` می‌پذیرد

`TRUNCATE`	همه ردیف‌های جدول	بله	در حالت معمول خیر
`DROP TABLE`	خود شیء جدول و ساختارش	خیر	خیر

- **تکمیل فنی:** عبارت «`DELETE` همیشه ردیف‌به‌ردیف و کند است» قانون همگانی نیست؛ طرح اجرا، Index, Trigger, Logging و محصول دیتابیس مهم‌اند. در PostgreSQL، `TRUNCATE` درون Transaction قابل Rollback است؛ بعضی سامانه‌ها رفتار دیگری دارند. قبل از استفاده در محیط واقعی، مستندات همان محصول را بررسی کنید.

۴.۴ DCL و TCL: دسترسی و تراکنش

مدرس مثال زد که یک کاربر تحلیلگر باید بتواند جدول موردنیاز را بخواند، اما نباید به همه جدول‌ها، مخصوصاً اطلاعات حساس پرداخت، دسترسی نامحدود داشته باشد. این اصل همان حداقل دسترسی (Least Privilege) است.

```
-- Illustrative only: the role must exist and permissions depend on DBMS
GRANT
SELECT ON customers TO analyst_role;
```

در بحث تراکنش، مدرس مثال ثبت سفارش و پرداخت را مطرح کرد: اگر دو تغییر به هم وابسته‌اند، نباید یکی ثبت شود و دیگری به علت خطا نیمه‌کاره بماند. دستورهای `BEGIN`، `COMMIT` و `ROLLBACK` مرز انجام مجموعه تغییرها را مشخص می‌کنند.

```
-- Demonstration on a disposable table
BEGIN;
UPDATE products_demo
SET
    unit_price = unit_price + 10
WHERE
    product_id = 61;
ROLLBACK;
```

- **تکمیل فنی:** موفقیت تراکنش به این معنا نیست که هر عملیات خارجی، مثل ارسال ایمیل یا فراخوانی API، خودکار همراه دیتابیس Rollback می‌شود. تراکنش معمولاً عملیات تحت کنترل همان موتور یا هماهنگ‌کننده تراکنش را پوشش می‌دهد.

۵. خواندن داده با SELECT

۵.۱ ساختار یک Query ساده

در نمایش کلاس، مدرس از جدول مشتریان شروع کرد، همه ستون‌ها را دید و سپس تعداد ردیف خروجی را محدود کرد.

```
SELECT
    *
FROM
    customers
LIMIT 4;
```

ستاره یعنی همه ستون‌های جدول؛ بهتر است در Queryهای نگهداری‌شونده، نام ستون‌های واقعاً موردنیاز را مشخص کنیم.

```
SELECT
    customer_id,
    full_name,
    country
FROM
    customers
WHERE
    country = 'Iran'
ORDER BY
    full_name ASC
LIMIT 5;
```

در این الگو `FROM` منبع را مشخص می‌کند، `WHERE` ردیف‌ها را محدود می‌کند، `ORDER BY` ترتیب نمایش را معین می‌کند و `LIMIT` تعداد خروجی را محدود می‌کند.

- **تکمیل فنی:** بدون `ORDER BY`، «اولین پنج ردیف» ترتیب تضمین‌شده‌ای ندارد. همچنین در برخی DBMSها `LIMIT` وجود ندارد؛ دستور متناظر باید متناسب با Dialect انتخاب شود.

۵.۲ فیلترها: AND, OR, IN, BETWEEN و LIKE

مدرس روی فیلتر هم‌زمان دسته محصول و بازه قیمت کار کرد؛ مثلاً محصولات دسته‌های Books یا Sports با قیمت بین دو حد.

```
SELECT
    product_name,
```

```
    category,
    unit_price
FROM
    products
WHERE
    category IN ('Books', 'Sports')
    AND unit_price BETWEEN 10
    AND 30
ORDER BY
    unit_price DESC;
```

در عبارت بالا، `IN` عضویت در فهرست، `AND` برقرار بودن هر دو شرط و `BETWEEN` بازه شامل ابتدا و انتهای آن را می‌سنجد. `LIKE` برای الگوی متنی و `NOT` برای نقیض شرط استفاده می‌شود. با ترکیب `AND` و `OR` بهتر است پرانتز بگذاریم تا اولویت منطقی مبهم نباشد.

- **تکمیل فنی:** `BETWEEN` برای تاریخ و زمان نیازمند توجه به کران‌هاست. اگر ستونی از نوع Timestamp باشد، کران بالای «آخر روز» ممکن است همه رویدادهای آن روز را شامل نشود. در آن شرایط معمولاً فیلتر نیمه‌باز، با `>=` برای شروع و `<` برای آغاز روز بعد، روشن‌تر است.

۶. توابع تجمیعی و گروه‌بندی

۶.۱ COUNT, SUM, AVG, MIN و MAX

وقتی سوال از «چند تا»، «جمع چقدر»، «میانگین چقدر» یا «بیشترین و کمترین» است، از Aggregate Function استفاده می‌کنیم. مدرس این توابع را مقدمه‌ای برای گزارش‌های تحلیلی معرفی کرد.

```
SELECT
    COUNT(*) AS customer_count
FROM
    customers;
```

`COUNT(*)` تعداد ردیف‌ها را می‌شمارد. در مقابل، `COUNT(city)` فقط مقدارهای غیر NULL ستون شهر را می‌شمارد. به این دلیل دانستن Grain داده و وضعیت NULL پیش از تفسیر آمار ضروری است.

۶.۲ GROUP BY: هر کشور چند مشتری دارد؟

مثال اصلی کلاس، شمارش مشتریان به تفکیک کشور بود.

```
SELECT
    country,
```

```
    COUNT(*) AS customer_count
FROM
    customers
GROUP BY
    country
ORDER BY
    customer_count DESC;
```

پس از گروه‌بندی، هر ردیف خروجی نماینده یک گروه کشور است، نه یک مشتری. ستون غیرتجمیعی `country` باید مطابق قواعد DBMS در `GROUP BY` حاضر باشد یا از نظر وابستگی تابعی قابل قبول باشد.

۶.۳ HAVING: فیلتر پس از گروه‌بندی

`WHERE` ردیف‌های ورودی را محدود می‌کند، اما `HAVING` شرطی روی گروه‌های محاسبه‌شده می‌گذارد؛ مثلاً کشورهایی را نگه می‌دارد که بیش از چهار مشتری دارند.

```
SELECT
    country,
    COUNT(*) AS customer_count
FROM
    customers
GROUP BY
    country
HAVING
    COUNT(*) > 4
ORDER BY
    customer_count DESC;
```

- **تکمیل فنی:** در بعضی سامانه‌ها می‌توان نام مستعار Aggregate را در `HAVING` به کار برد و در برخی نمی‌توان. تکرار عبارت `COUNT(*)` در این مثال از نظر سازگاری، انتخاب مطمئن‌تری است.

۷. ترتیب نوشتاری و ترتیب منطقی پردازش

یکی از تأکیدهای اصلی مدرس این بود که Query با `SELECT` نوشته می‌شود، اما از دید **ترتیب منطقی**، دیتابیس ابتدا منبع و ردیف‌های واجد شرایط را تعیین می‌کند و بعد نتیجه انتخاب و نمایش ساخته می‌شود.

جایگاه نوشتاری	بخش	جایگاه در مدل منطقی ساده‌شده
۱	`SELECT`	پس از فیلتر و گروه‌بندی
۲	`FROM` / `JOIN`	ابتدا
۳	`WHERE`	پس از تعیین منبع

۴	`GROUP BY`	پس از فیلتر ردیف
۵	`HAVING`	پس از ساخت گروه
۶	`ORDER BY`	پس از تعیین خروجی
۷	`LIMIT`	پس از مرتب‌سازی

- **تکمیل فنی:** این جدول ترتیب **منطقی** فهم SQL است، نه ترتیب واقعی اجرای فیزیکی دستورات توسط پردازنده. Optimizer می‌تواند فیلترها، Joinها و سایر عملیات را در Execution Plan بازآرایی کند، اگر نتیجه معنایی Query حفظ شود.

مدرس نمونه‌ای زد که در `SELECT` نام مستعار `net_amount` ساخته شده، ولی همان نام در `WHERE` قابل استفاده نیست؛ علت همین ترتیب منطقی است.

```
-
- The expression is repeated because its SELECT alias is not visible in WHERE
SELECT
    order_id,
    order_amount * 0.9 AS net_amount
FROM
    orders
WHERE
    order_amount * 0.9 > 100;
```

۷.۱ CASE و COALESCE برای داده خواناتر

`CASE` یک بیان شرطی می‌سازد؛ مثلاً محصولات را به رده‌های قیمت تقسیم می‌کند. `COALESCE` اولین مقدار غیر NULL را انتخاب می‌کند و در مثال کلاس، برای نشان‌دادن برچسب قابل‌فهم به جای کشور نامشخص استفاده شد.

```
SELECT
    product_name,
    unit_price,
    CASE
    WHEN unit_price < 20 THEN 'Low'
    WHEN unit_price < 50 THEN 'Medium'
    ELSE 'High'
END AS price_band
FROM
    products;
```

```sql
SELECT
    COALESCE(country, 'Unknown') AS country_label,
    COUNT(*) AS customer_count
FROM
    customers
GROUP BY
    COALESCE(country, 'Unknown');
```

- **تکمیل فنی:** جایگزین‌کردن NULL با `Unknown` برای گزارش‌گیری با تغییر خودِ مقدار ذخیره‌شده در جدول فرق دارد. همچنین اگر واقعاً مقداری به نام `Unknown` در داده وجود داشته باشد، تجمیع آن با NULL ممکن است باعث ادغام دو وضعیت متفاوت شود.

۸. JOIN: ترکیب جدول‌ها و مسئله معنای ردیف

۸.۱ چرا جدول‌ها را جدا ذخیره می‌کنیم؟

مدرس توضیح داد در مدل رابطه‌ای اطلاعات مشتری یک بار در `customers` نگهداری می‌شود، ولی در جدول `orders` شناسه همان مشتری ثبت می‌شود. برای گزارش نام مشتری کنار شماره سفارش، باید این دو را بر اساس کلید مرتبط به هم وصل کنیم. این عمل `JOIN` است.

نوع JOIN	چه ردیف‌هایی را نگه می‌دارد؟
`INNER JOIN`	جفت‌هایی که شرط اتصال را برآورده کنند
`LEFT JOIN`	همه ردیف‌های سمت چپ؛ ردیف‌های بدون جفت با NULLهای سمت راست
`RIGHT JOIN`	همه ردیف‌های سمت راست؛ ردیف‌های بدون جفت با NULLهای سمت چپ
`FULL OUTER JOIN`	همه ردیف‌های دو طرف و جفت‌های متناظر

- **تکمیل فنی:** `JOIN` صرفاً به ستون‌هایی با نام یکسان محدود نیست؛ شرط اتصال می‌تواند از ستون‌هایی با نام متفاوت یا حتی شرط‌های دیگری تشکیل شود. در بعضی سامانه‌ها `CROSS JOIN` بدون `ON` وجود دارد و همه ترکیب‌های دو جدول را تولید می‌کند. نوع و شرط JOIN باید با هدف گزارش و رابطه منطقی داده‌ها سازگار باشند.

۸.۲ مثال مشتری و سفارش

مثال اصلی کلاس، نگه‌داشتن همه مشتریان و پیوست‌کردن سفارش‌های آنان بود.

```
SELECT
    c.customer_id,
    c.full_name,
    o.order_id
FROM
    customers AS c
LEFT JOIN orders AS o
    ON o.customer_id = c.customer_id;
```

در این Query اگر یک مشتری پنج سفارش داشته باشد، پنج ردیف به او اختصاص پیدا می‌کند. اگر هیچ سفارشی نداشته باشد، یک ردیف با `order_id` برابر `NULL` خواهد داشت. بنابراین «هر ردیف خروجی نماینده چیست؟» پاسخ متفاوتی با جدول اولیه مشتریان دارد.

مدرس در نمایش کلاس از ۵۰٬۰۰۰ سفارش و ۷۵۰ مشتری بدون سفارش یاد کرد. با فرض درست‌بودن این اعداد و اینکه هر سفارش دقیقاً با یک مشتری تطبیق یابد، `LEFT JOIN` از سمت مشتریان **۵۰٬۷۵۰** ردیف می‌سازد.

- **تکمیل فنی:** این ۵۰٬۷۵۰ ردیف به معنی ۵۰٬۷۵۰ سفارش نیست؛ ۵۰٬۰۰۰ ردیف متناظر با سفارش‌اند و ۷۵۰ ردیف، مشتریان بدون سفارش را نمایش می‌دهند. این برابری به یکتایی کلید مشتری و وجود رابطه درست هر سفارش با یک مشتری وابسته است. اگر کلید سمت مقابل تکراری یا JOIN نادرست باشد، تعداد ردیف‌ها می‌تواند افزایش ناخواسته پیدا کند.

۸.۳ یافتن مشتریان بدون سفارش

```
SELECT
    c.customer_id,
    c.full_name
FROM
    customers AS c
LEFT JOIN orders AS o
    ON o.customer_id = c.customer_id
WHERE
    o.order_id IS NULL;
```

مثال اضافی: راه دیگری برای همان هدف، استفاده از `NOT EXISTS` است. این روش به‌ویژه برای بیان «برای این مشتری هیچ سفارش متناظری وجود ندارد» خوانا است:

```sql
SELECT
    c.customer_id,
    c.full_name
FROM
    customers AS c
WHERE
    NOT EXISTS (
    SELECT
        1
    FROM
        orders AS o
    WHERE
        o.customer_id = c.customer_id
);
```

۸.۴ چرا پس از JOIN شمارش را دوباره بررسی می‌کنیم؟

برای گزارش «تعداد مشتریان»، `COUNT(*)` روی خروجی JOIN ممکن است به‌جای مشتریان، ردیف‌های حاصل از سفارش‌ها و مشتریان بدون سفارش را بشمارد. بسته به سوال می‌توان از `COUNT(DISTINCT c.customer_id)` برای شمارش مشتریان یکتا یا از `COUNT(o.order_id)` برای شمارش سفارش‌های متناظر استفاده کرد.

```sql
SELECT
    COUNT(DISTINCT c.customer_id) AS customers_count,
    COUNT(o.order_id) AS matched_orders_count
FROM
    customers AS c
LEFT JOIN orders AS o
    ON o.customer_id = c.customer_id;
```

- **تکمیل فنی:** پیش از JOIN باید Grain، یکتایی کلیدها و Cardinality هر جدول مشخص باشد. اتصال اشتباه جدول‌هایی با چند ردیف به‌ازای یک کلید می‌تواند Fan-out, Double Counting و بزرگ‌شدن نادرست Metric ایجاد کند. نمونه عددی بالا فقط برای توضیح منطقی کلاس است، نه نتیجه آزمایش مستقل روی دیتاست.

۹. موارد اشاره‌شده برای ادامه یادگیری

در پایان Transcript به امکان `UNION`، `EXISTS`، `NOT EXISTS`، اتصال یک جدول با خودش (Self Join) و CTE اشاره شد، ولی آموزش کامل و مثال عملی همه این موارد در فایل ارسالی ثبت نشده است. قرار است در مباحث بعدی، این ابزارها در ترکیب Queryهای بزرگ‌تر روشن‌تر شوند.

۱۰. Workflow تمرین و چک‌لیست

۱. دیتابیس آزمایشی فروشگاه را باز کنید و نام جدول‌ها و ستون‌ها را بررسی کنید.

۲. پیش از هر Query بنویسید «هر ردیف این جدول نماینده چیست؟».

۳. Query ساده با `SELECT` و `WHERE` بنویسید و وضعیت `NULL` را بررسی کنید.

۴. با `COUNT(*)` تعداد ردیف‌ها را بشمارید و سپس `GROUP BY` و `HAVING` را امتحان کنید.

۵. قبل و بعد از هر JOIN تعداد ردیف و تعداد کلیدهای یکتا را با هم مقایسه کنید.

۶. `UPDATE`، `DELETE` و تغییر DDL را فقط در محیط آزمایشی و با Transaction یا امکان بازگردانی اجرا کنید.

۷. برای هر تفاوت نتیجه، ابتدا تعریف Metric، شرط‌ها، داده‌های گمشده و رابطه جدول‌ها را بررسی کنید.

۱۱. جمع‌بندی

در این جلسه مشخص شد SQL، زبان مشترک بیان خواسته‌های ما از یک DBMS است. ساختار جدول و کلیدها، انواع دستورها، مفهوم تراکنش و مجوز، فیلتر، توابع Aggregate و ترتیب منطقی پردازش SQL به هم وابسته‌اند. مهم‌ترین نقطه عملی جلسه JOIN بود: بدون درک ارتباط کلیدها و معنای هر ردیف، حتی Queryای که از نظر Syntax درست اجرا شده می‌تواند گزارش اشتباه تولید کند.

۱۲. سوال‌های مرور

۱. چرا برای یافتن مجموع خرید سارا در یک فایل متنی ساده باید برنامه جداگانه نوشت؟

۲. تفاوت زبان SQL و نرم‌افزار DBMS چیست؟

۳. چرا SQL را Declarative می‌نامند؟

۴. Primary Key در جدول مشتریان چه وظیفه‌ای دارد؟

۵. تفاوت `NULL`، صفر و رشته خالی چیست؟

۶. `DELETE`، `TRUNCATE` و `DROP TABLE` چه تفاوتی دارند؟

۷. برای محدودکردن دسترسی یک تحلیلگر به جدول خاص از چه خانواده دستورهایی استفاده می‌شود؟

۸. چرا `WHERE order_amount * 0.9 > 100` معتبر است ولی استفاده مستقیم از نام مستعار ساخته‌شده در `SELECT` معمولاً در `WHERE` معتبر نیست؟

۹. تفاوت `COUNT(*)` و `COUNT(city)` چیست؟

۱۰. چه زمانی باید از `HAVING` به‌جای `WHERE` استفاده کنیم؟

۱۱. در یک `LEFT JOIN` اگر مشتری سه سفارش داشته باشد چند ردیف متناظر می‌سازد؟

۱۲. اگر مشتری سفارش نداشته باشد، نتیجه `LEFT JOIN` از سمت `customers` چگونه است؟

۱۳. چرا تعداد خروجی JOIN الزاماً برابر تعداد مشتری یا سفارش نیست؟

۱۴. برای فهرست مشتریان بدون سفارش، چه شرطی روی `LEFT JOIN` مناسب است؟

۱۵. اگر مجموع فروش پس از JOIN دو برابر شد، نخست چه مواردی را بررسی می‌کنید؟

۱۳. پاسخ‌نامه مختصر

۱. چون خواندن فایل، شکستن خط‌ها، تبدیل مبلغ و جمع‌زدن باید در منطق برنامه پیاده شود.

۲. SQL زبان بیان Query و عملیات داده است؛ DBMS نرم‌افزار مدیریت اجرای عملیات و نگهداری داده است.

۳. چون نتیجه موردنیاز را توصیف می‌کند، نه همه جزئیات اجرای فیزیکی را.

۴. یکتایی و غیر NULL بودن شناسه هر مشتری را تضمین می‌کند.

۵. `NULL` فقدان مقدار شناخته‌شده است؛ صفر عدد است و رشته خالی یک مقدار متنی.

۶. حذف ردیف‌های انتخابی، پاک‌سازی همه ردیف‌ها با باقی‌ماندن جدول، و حذف خود جدول.

۷. DCL، مانند `GRANT` و `REVOKE`.

۸. چون `WHERE` در مدل منطقی پیش از ساخت ستون مستعار `SELECT` پردازش می‌شود.

۹. اولی تمام ردیف‌ها و دومی فقط شهرهای غیر NULL را می‌شمارد.

۱۰. وقتی شرط روی حاصل Aggregate یا گروه‌هاست.

۱۱. سه ردیف، به شرط اینکه هر سفارش یک تطبیق معتبر داشته باشد.

۱۲. یک ردیف مشتری همراه با `NULL` در ستون‌های جدول سفارش.

۱۳. چون رابطه یک‌به‌چند یا چندبه‌چند تعداد ترکیب‌های ردیفی را تغییر می‌دهد.

۱۴. `WHERE o.order_id IS NULL`، با فرض کلید سفارش غیر NULL.

۱۵. Grain، یکتایی Key, Cardinality، شرط JOIN و تکرار رکوردها را بررسی می‌کنیم.

۱۴. واژه‌نامه

ردیف	اصطلاح	توضیح کوتاه
۱	SQL	زبان استاندارد پرس‌وجو و کار با داده
۲	DBMS	سامانه مدیریت پایگاه داده
۳	Relational Database	پایگاه داده بر مبنای جدول و رابطه
۴	Query	درخواست خواندن یا تغییر داده
۵	Declarative	بیان نتیجه موردنیاز به‌جای مراحل اجرای فیزیکی
۶	Row	یک ردیف داده با معنای مشخص
۷	Column	ویژگی یا فیلد داده
۸	Primary Key	شناسه یکتا و غیر NULL
۹	Foreign Key	کلید ارجاع به رکورد مرتبط
۱۰	Constraint	قاعده اعتبار داده
۱۱	Transaction	مجموعه عملیات با مرز تأیید یا بازگشت
۱۲	DDL	تعریف و تغییر ساختار
۱۳	DML	افزودن، ویرایش و حذف داده
۱۴	DQL	خواندن و پرس‌وجوی داده
۱۵	DCL	مدیریت دسترسی
۱۶	TCL	کنترل تراکنش
۱۷	NULL	فقدان مقدار شناخته‌شده
۱۸	WHERE	فیلتر ردیف‌های ورودی
۱۹	GROUP BY	گروه‌بندی ردیف‌ها
۲۰	HAVING	فیلتر گروه‌های حاصل
۲۱	Aggregate	محاسبه خلاصه مانند جمع و شمارش
۲۲	JOIN	ترکیب ردیف‌ها بر اساس شرط ارتباط

ردیف	اصطلاح	توضیح کوتاه
۲۳	Cardinality	تعداد و نوع تطبیق ردیف‌ها در رابطه
۲۴	Grain	معنای دقیق هر ردیف جدول
۲۵	CTE	عبارت موقت نام‌دار برای سازمان‌دهی Query

۱۵. انتقال تجربه مدرس

مدرس چند بار هشدار داد که اجرای دستور درست به معنای سالم‌بودن نتیجه نیست. در محیط Production باید با `ALTER`، `DELETE`، `DROP` و هر تغییر گسترده با احتیاط کار کرد. در تحلیل نیز مهم است Query را با نیاز واقعی کسب‌وکار تطبیق دهیم و درباره تعداد ردیف‌ها، معنی هر جدول و رفتار `NULL` حساس باشیم. او همچنین به اهمیت تمرین، کاربرد SQL در مصاحبه‌های فنی و محدودیت زمان برای پوشش همه مفاهیم اشاره کرد.